Advanced Machine Learning Coursework: Optimising Cryptocurrency Trading Strategies Under Fee Constraints with Deep Deterministic Policy Gradients

March 25, 2025

1 Introduction

1.1 Motivation and Background

Cryptocurrency markets have become a focal point of modern finance, attracting traders and data scientists alike due to their volatile nature and potential for high returns (Liu et al., 2022). However, despite efforts to manufacture a profitable trading strategy, market participants are met with the harsh reality of trading costs and complex fee structures that dent their well-earned theoretical profits. The following paper investigates how Deep Deterministic Policy Gradients (DDPG), a Reinforcement Learning (RL) method, can optimise cryptocurrency trading under real-world fee constraints in the dynamic cryptocurrency market environment.

RL is well-suited to this domain, as agents learn through sequential decision-making in dynamic environments, closely mirroring real-market conditions (Winder, 2020). Unlike static rule-based systems, RL agents adapt their strategies over time, optimising directly for returns in changing market states (Deng et al., 2017).

The DDPG algorithm has been chosen specifically for this experiment as it can operate in higher dimension state spaces and output continuous actions, allowing precise trade position sizing — such as buying 0.01 BTC — rather than discrete decisions like "buy" or "sell" (Zhang et al., 2021; Tummon et al., 2020). By maximising a reward function that includes transaction costs, DDPG enables agents to weigh potential gains against fees, reflecting real-world situations and profitability (Winder, 2020). Previous research has demonstrated that DDPG can be effectively applied to various trading and portfolio management tasks, often outperforming a baseline Buy-and-Hold strategy — an outcome that this paper also aims to achieve.

1.2 Literature Review

The following section provides a focused review of relevant research on RL, particularly DDPG, within the context of finance and trading. It first provides a brief overview of cryptocurrency exchange fees followed by a background to DDPG. Finally the section covers relevant applications of DDPG including how DDPG has augmented models, the practical challenges researchers have faced, including any transaction costs modelled.

1.2.1 Transaction costs and fees

Cryptocurrency exchanges charge fees for interacting with the exchange – deposits, withdrawals, buying, selling and converting cryptocurrencies can all incur costs. For some of these fees, platforms charge a flat rate on the value of asset traded or exchanged (usually expressed as a percentage of the underlying transaction). Other platforms encourage traders to contribute liquidity to the exchange by employing a maker-taker fee structure with tiered trade volume fee discounts. For example, taker fees can range from 0.001% to 0.6% depending on the 30 day trading volume and the product traded,

versus 0.0% to 0.4% fees for a high-volume maker (See Kraken, 2025; Coinbase, 2025; Binance, 2025 etc.). [1]

Felizardo, et al. (2022) note that transaction costs are now considered essential in RL simulations, where only 5 of 29 studies surveyed omitted transaction fees. Costs were typically modelled as either as a percentage or fixed value per trade (Felizardo, et al., 2022).

1.2.2 Background to DDPG

As a foundation to the analysis of DDPG literature and the methodology of this paper, a breif history to the DDPG. The DDPG algorithm, presented by Lillicrap et al. (2016), includes key elements of the Deterministic Policy Gradient (DPG) method (Silver et al., 2014) and integrates stabilisation techniques from Deep Q-Networks (DQN), including experience replay and target networks (Mnih et al., 2015). Experience replay forms part of the learning process for an agent where past state transitions form a buffer, breaking the temporal correlation in online learning. Target networks, updated via "soft" updates, reduce instability by ensuring that the targets used to train the critic evolve gradually over time (Lillicrap et al., 2016).

Unlike traditional reinforcement learning methods such as Q-learning, which struggle with continuous and high-dimensional action spaces (Lillicrap et al., 2016), DDPG uses deterministic policy gradients to enable efficient learning in such environments by directly mapping states to actions (Silver et al., 2014). This combination of deterministic policy updates, experience replay, and target networks facilitates stable training while managing the complexity of continuous control tasks (Sliver et al., 2014)(Lillicrap et al., 2016). As a result, DDPG is particularly well-suited to applications such as algorithmic trading, where agents must learn to make fine-grained trading decisions in dynamic, high-dimensional environments.

1.2.3 DDPG applications

DDPG has been applied to a range of trading tasks in both traditional and crypto markets. For example, Tummon et al. (2020) applied DDPG to the ETH/USD perpetual swaps market on BitMEX, leveraging the continuous action space to enable fractional position sizing. Their best-performing model was able to outperform a buy-and-hold benchmark, particularly during volatile market conditions. While results varied depending on the feature set and training window, the findings illustrate DDPG's potential to identify short-term trading opportunities and adjust exposure dynamically. However, the study did not explicitly incorporate transaction costs, and the authors noted a risk of over fitting to historical price patterns (Tummon et al., 2020).

Zhang et al. (2021) applied DDPG to long-term US equity trading, enhancing the model with LSTM-based critic networks to capture temporal dependencies. Their model included transaction fees (fixed at $0.001 per trade), and they found that this improved overall performance and robustness. By discouraging excessive trading, transaction costs served as a form of regularisation, encouraging more stable, long-term strategies. Their DDPG model outperformed both baseline and fee-free variants in terms of Sharpe ratio and cumulative return, even across turbulent market periods such as the 2008 financial crisis and the COVID-19 crash (Zhang et al. 2021).

A more recent study by Mastrogiovanni (2024) evaluated four DDPG variants using different reward functions, including cumulative return, expected shortfall, and drawdown control, in a dynamic cryptocurrency portfolio management setting. The model without change penalty achieved the highest cumulative return (0.90), annual return (0.56), and Sharpe ratio (1.51), but incurred higher tail risk (VaR 95% = -0.030). In contrast, the model with a penalty for frequent portfolio changes showed lower returns (cumulative return = 0.45) but offered better tail-risk management and lower transaction costs. Other variants optimising for minimum drawdown and expected shortfall provided more balanced risk-adjusted results. The paper highlights how DDPG can be customised through its reward function to suit different objectives.

[1] There are many additional trading cost constraints in the cryptocurrency market from depositing and withdrawing assets from an exchange, network fees such as gas fees or congestion costs, trading costs such as slippage in price when a market moving trade is placed (yielding a difference in expected and actual price) and spreads between the bid and ask price etc. While these costs are not analysed in this paper, they are all factors that contribute to the final price of an asset or net worth of a portfolio.

1.3 Research Direction

The following research looks to build on the existing DDPG literature by enriching the state space with market features, capturing temporal patterns through the neural network structure and, most importantly, including a range of trading fees in the agent reward. This research seeks to advance the practical application of DDPG models for cryptocurrency trading in real-world scenarios where transaction costs significantly impact performance, with a view to optimising the DDPG models such that they outperform a baseline buy-and-hold strategy.

2 Methodology

The following section presents the mathematical formulation and implementation of the DDPG model. The methodology includes the framework of the DDPG architecture, the neural network model for the actor and critic, the sequential replay buffer design for time series data, the state and action space design, the reward function and transaction costs along with key modelling assumptions. Finally, the methodology ends with a brief description of the cryptocurrency data plus exploratory data analysis. A simple baseline strategy is also introduced as a comparative benchmark.

2.1 DDPG algorithm

DDPG is an off-policy, actor-critic approach for continuous action spaces (Lillicrap et al., 2016). DDPG contains elements DPG theory (Silver et al., 2014)(Lillicrap et al., 2016), which allows for learning a deterministic policy.

To intuitively explain the DDPG algorithm, consider an analogy to a trading desk. The actor is akin to a junior trader who proposes trading decisions based on observed market conditions (the state s), while the critic resembles a Head of Desk who evaluates those decisions and provides feedback. The actor is a neural network that learns a deterministic policy, given by $\mu(s|\theta^\mu)$, and maps states to actions, that is, which trade to execute in a given market environment (Lillicrap et al., 2016).

The actor's proposed action a is passed to the critic, a separate neural network, that estimates the expected return via the Q-value function $Q(s, a|\theta^Q)$ (Lillicrap et al., 2016). This value reflects the long-term reward predicted by taking action a in the state s. The actor is trained to select actions that maximise these Q-values, effectively learning to take actions the critic evaluates as valuable (Silver et al., 2014).

2.1.1 Actor updates

As noted, the actor's goal is to maximise the expected return by producing actions that the critic evaluates as high-value. This is achieved by applying the DPG theorem (Silver et al., 2014). Unlike stochastic policy gradient methods, which integrate over both state and action distributions, DPG simplifies the gradient computation by integrating over the state space only. This improves learning efficiency in continuous action spaces (Silver et al., 2014). The DPG theorem states that the gradient of the expected return can be estimated using the critic's gradient with respect to the action, combined with the actor's own gradient. This gradient update guides the actor to adjust its parameters θ^μ so it produces actions that the critic rates highly, leading to greater cumulative reward and higher Q-values — in effect, the actor is learning to behave in ways that the critic deems more profitable (Silver et al., 2014).

This formulation is a key distinction of DDPG as it allows deterministic, efficient, and scalable learning in high-dimensional action spaces, such as those encountered in financial trading (Silver et al., 2014).

2.1.2 Critic updates

The critic is trained by minimising the Bellman error. The Bellman error is the difference between the predicted Q-values and target Q-values (Mnih et al., 2015). Given a transition (s_t, a_t, r_t, s_{t+1}), the target value is (Bhatia et al., 2019):

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'}$$

where Q' and μ' are the target networks (discussed below) and γ is the hyperparameter discount factor. The critic network is trained to minimise the Mean Squared Error (MSE) between the predicted and target Q-values. In effect, the critic is updated by minimising the difference between what the critic predicts and the target. Minimising this loss via gradient descent aligns the critic's predictions with more accurate targets, improving its ability to evaluate the actor's decisions and provide reliable feedback to the actor.

2.1.3 Exploration noise

As the actor outputs deterministic actions, explicit noise must be added during training to encourage exploration. Decaying Gaussian noise, drawn from a normal distribution has been applied:

$$\epsilon_t \sim \mathcal{N}(0, \sigma_t^2), \quad \sigma_t = \max(0.05,\ 0.5 \cdot 0.995^{\text{episode}})$$

This promotes broad exploration early in training, gradually shifting toward more stable actions as the policy improves. The noisy action is clipped to [-1, 1] to ensure the action stays within the valid range. Such an approach is simpler than temporally correlated Ornstein–Uhlenbeck noise and is effective for continuous control tasks like trading. Noise is removed during evaluation to allow deterministic policy execution.

2.1.4 Target networks

DDPG employs target networks for both actor and critic, as introduced by Mnih et al. for DQN and adapted by Lillicrap et al. for DDPG, to improve stability. Both target networks are updated using a soft update mechanism:

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$$

(from Lillicrap et al. 2016) τ is a small number (0.005 in this model), making the target networks change slowly. This provides stability to the learning process. This means the target networks change only gradually (Lillicrap et al. 2016), so the target y_t in the critic loss is computed using slowly moving parameters. This technique greatly improves training stability (Lillicrap et al. 2016) by reducing divergence that can occur if the critic target is based on rapidly changing estimates.

2.2 Actor and critic network model

The actor and critic are deep neural networks, implemented using TensorFlow with two hidden layers each. The structure of the neural networks follows a similar framework to the previously discussed DDPG papers (see Lillicrap et al. (2016) and Mastrogiovanni (2024)) and are tailored to the dimensions of the required DDPG inputs and outputs.

The actor network uses two fully connected ReLU layers followed by a tanh activation to output actions bounded in [-1, 1]. The critic network inputs the state-action pair and returns the Q-value. Layer sizes are 164 and 64 neurons for the actor network and 128 and 64 neurons for the critic network. A dropout regularisation was added after initial runs to help the model generalise. Both networks are trained using Adam optimisers and standardised inputs. During the training, the critic and actor are updated using batches sampled from the replay buffer as set out below.

2.3 Replay Buffer

The replay buffer stores experiences in the form of state transitions in the tuple (s_t, a_t, r_t, s_{t+1}). Traditional DDPG assumes independent training samples however and samples on a mini-batch basis to remove correlations (Mnih et al., 2015). To preserve temporal dependencies in financial time series, a Sequential Replay Buffer samples sequences of 10 hourly transitions instead of random experiences. This helps the model learn market patterns like momentum and volatility (Zhang et al., 2020).

2.4 Action space

The action space A_t is continuous, allowing for fractional trades $A_t \in [-1, 1]$. Actions are continuous within [-1, 1]: positive for buying, negative for selling. Trade sizes are capped at 50% of available holdings or cash, to reduce erratic trades undertaken by the agent, and each trade incurs transaction fees.

The action formulation allows for specific and fine-grained actions as opposed to discrete "buy/ sell/ hold" positions. Note that all positions must be funded by existing capital and holdings. The agent can never sell or buy more BTC than it currently has available in the portfolio.

2.5 Reward function

The reward is defined as r_t to directly reflect the change in holdings value due to the agent's action and the subsequent price movement, net of transaction costs. V_t is the total net value at time t and C_t the transaction cost.

$$r_t = (V_t - V_{t-1}) - C_t$$

In summary, the reward represents the net change in wealth over the last hour (Spinning Up, 2025). The reward encourages the agent to grow its portfolio value but penalises excessive trading through the cost element. No explicit risk term is added to the reward, but indirectly, the agent may learn to control risk because large volatile positions can lead to negative rewards in down moves. Transaction costs in the reward ensure the agent is aware of the penalty for frequent trading, promoting a more cost-efficient strategy.

2.6 Transaction costs

As noted above, this paper looks to address the impact of different fee structures on the DDPG agents trading strategy. Three cost structures have been incorporated into the reward as follows:

1. **No transaction costs** - No costs or penalties associated with trading decisions. The scenario should provide a baseline for if the agent can make profitable decisions without any constraints.

2. **Flat fee rate** - A scenario with a constant transaction cost of 0.1% for each trade, as a percentage of the trade amount. The scenario penalises every transaction proportionally and emulates exchanges that charge a flat percentage without volume discounts (per below). A slight amendment to the first scenario where the agent will need to adapt to constant constraints.

3. **Tiered taker fees** - A fee schedule resembling some of the more mainstream crypto exchanges maker-taker models. The fee rate depends on the trading volume over a 30 day rolling window, where discounts are offered for higher volume traders. Note for the purposes of this experiment we will analyse 'taker' fees. The scenario will test to see how the agent reacts to a more realistic trading environment - where initially low volumes of trading will incur higher costs, which may encourage the agent to increase volumes of trade to reach the more cost-effective higher tiers.

So as not to disproportionately promote a single exchange, a table of "taker" fees resembling a range of main crypto exchanges has been created. By modelling each of the three transaction cost scenarios, this paper will see how robust the training of the agent is and how its actions change under constant and variable constraints.

2.7 Trading environment data

The environment has been modelled on historical, hourly BTC/ USD price data. The date ranges from from 15th August to 15th October 2024 with training date ranges from 15th August to 30th September 2024 and testing date ranges from 1st October to 15th October. In total, there are 1,128 timestamps for the training phases and 336 timestamps for testing phases.

The period has been chosen as the time frame has both peaks and troughs that should provide the model with sufficient data to learn both downward and upward trends. Note however that Bitcoin was in a recovery phase following all-time highs in March 2024, where BTC/USD prices hit around $70,000. The rally may be in part due to Bitcoin halving effects (where the "Bitcoin network halves

30-day volume (USD)	Fee
0-10,000	0.3%
10,000-50,000	0.25%
50,000-100,000	0.2%
100,000-250,000	0.15%
250,000-500,000	0.1%
500,000-1,000,000	0.08%
1,000,000-2,500,000	0.06%
2,500,000-5,000,000	0.04%
5,000,000-10,000,000	0.02%
10,000,000+	0.001%

Table 1: Tiered taker fees.

Figure 1: Trading environment data - BTC/USD close price (1h interval)

the reward given to miners for processing transactions and adding new blocks to the blockchain") in April 2024 (Niklaus, 2025). Bitcoin subsequently dropped to $50,000 during the summer months. The August to October 2024 time frame showed Bitcoin regaining momentum, with prices climbing back toward the $60,000 range.

2.8 State Space and Feature Engineering

The price data extracted includes open, high, low, close and trading volume metrics for each hour. In order to create a state space that mirrors the financial market state and trading environment, features have been included that have been inspired by Zhang et al. (2020). With this in mind, several technical indicators have been added to the dataset to achieve a more full state space and provide signals to the agent as to making informed trades. The first correlation heatmap below shows all potential features analysed, the second correlation heatmap shows the final state space after removing unnecessary features.

The initial state space was cleansed as follows - first highly correlated features were removed. Open, High, Low, Close, SMA 10, and SMA 50 were all highly correlated (c.1.0 correlation coefficient - almost perfect positive correlation). Only the Close price was retained.

Moving Average Convergence Divergence, "MACD", which indicates trend direction and "SMA diff" (the difference between a 10 and 50 day simple moving average of close prices) with are also highly correlated (0.96) and so have been combined into one indicator called "Trend strength".

There are strong relationships between "Momentum" and "LogReturn 5" (0.69) and RSI (0.82). RSI Relative Strength Index, which captures momentum is also strongly correlates with MACD (0.85). As these features might be capturing similar movements only "LogReturn 5" feature was retained.

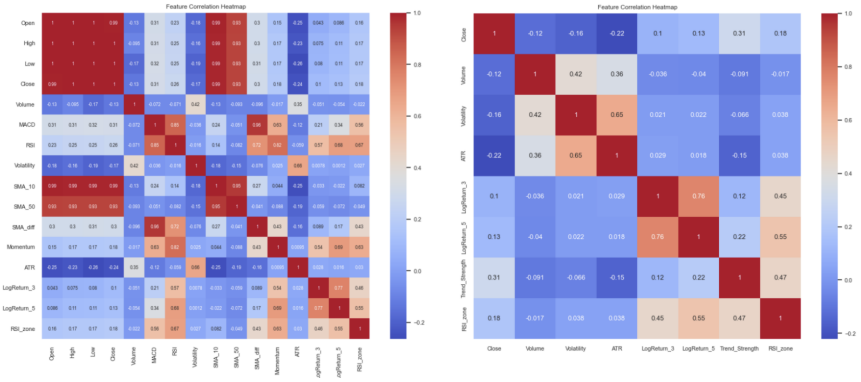

Figure 2: State space feature correlation heatmaps

There are weaker correlations between "SMA diff" and volume (-0.075), MACD vs Volume (–0.086), MACD vs ATR (–0.17) and "RSI" the Relative Strength Index vs ATR (–0.092). These features might provide more unique signals.

The final state space therefore includes:

- Close p_t – the closing price of BTC/USD in hourly intervals.
- Volume v_t– indicating levels of market activity and weakly correlated with the price.
- Trend strength TS_t– combination of MACD a momentum/ trend feature that should provide strong trading signals and SMA – the difference between a simple moving average for 10 and 50 periods. Indicates a trend direction.
- Volatility σ_t – for providing risk awareness.
- ATR ATR_t – range risk/ volatility signal
- Log return 3 and 5 $r_t^{(3)}$, $r_t^{(5)}$ – short and medium return value for forward looking momentum.
- RSI zone $RSIzone_t$ – indicating how over/ under bought the asset is. 1 if RSI ¿ 70 (overbought)-1 if RSI ¡ 30 (oversold), 0 otherwise

Therefore, at each time step t, the state vector is:

$$S_t = [p_t, v_t, TS_t, \sigma_t, ATR_t, r_t^{(3)}, RSIzone_t, {}_t^{(5)}]$$

Note that as a result of creating features with rolling windows, rows with null values were generated. Given the small number of impacted rows across the wider dataset, these rows were dropped resulting in a reduction to 1080 timestamps and eight features for training.

The features were also analysed for extreme values and were scaled using a standard scaler, ensuring each input to the neural network has zero mean and unit variance:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

. This should help to avoid a vanishing or exploding gradient. The scaler was fit on the training data and applied to the test data (excluding the RSI in both cases).

Each episode is comprised of hourly time steps that provides the agent with the state which would yield an action. The overall episode is then evaluated for overall metrics. The initial balance is then reset at the start of the next episode to provide a consistent baseline of performance.

2.9 Benchmark strategy: Buy and hold

The benchmark strategy used in this paper will be a buy and hold strategy. At the start of the period, the initial capital will be fully invested in BTC and will not trade in or out of this position throughout the period. The value of the strategy will mirror BTC/USD prices with minimal transaction costs. The benefit of using this strategy is that it provides a relatively simple benchmark to compare the robustness of our DDPG agent in comparison to a passive strategy. Such a strategy has also been deployed in other literature reviewed as a simple benchmark. The three models will be compared with the baseline strategy using net profits and cumulative returns cumulative returns as the basis for performance. Notably, as the market is generally upward trending, this strategy would perform well as it tracks the positive trend of the market (it would conversely track downward if in a bear market) - it might therefore be a tough benchmark to beat!

2.10 Assumptions

To focus this review on the main experiment, assumptions and model simplifications have been set out below:

- N**o leverage** - the agent can not borrow money or BTC to leverage its positions and is contained by the capital available. Any short positions are fully backed by the agents cash balance. The purpose of this assumption is to simplify risk management through removing margin calls from the modelling and reflects a conservative trading strategy.
- **No blockchain transaction fees** - It is assumed here that the trading occurs on a centralised exchange with no blockchain network fees (including network fees, gas fees or other on-chain transaction fees).
- **No slippage** - It is assumed that the agent can transact at the hourly close price and that there is sufficient liquidity such that the trades do not move the market. The assumption is to avoid including modelling with an order book.
- **No interest** - It is assumed that when the agent is holding cash, it yields no interest. The net worth is purely derived from the trading actions of the actor and BTC price changes.

Under these assumptions and using the methodology set out above, the DDPG model was trained within the described environment.

3 Results

The following section presents and analyses the results from the three DDPG trading strategies including No Fees, Flat Fee and Tiered Fees strategies. The baseline Buy-and-hold strategy is also included in the comparison. The section contains an analysis of the hyperparameter tuning and cross-validation steps applied to the initial results, prior to running the final model.

3.1 Analysis of final model

There are already some very interesting and unexpected results in the results of the experiment, as set out below:

In a no-fee environment, the DDPG agent just outperformed the baseline buy-and-hold strategy, with a final net worth of $108,330.79 versus $108,306.96, with an 8.33% return and over 59,000 trades executed. However, this success is shadowed by the agent's high-frequency trading strategy which resulted in a win rate of just 17.05%, indicating reliance on small, frequent gains — an approach that is highly vulnerable to transaction costs.

This vulnerability is confirmed when introducing fees. Under a flat fee of 0.001% per trade, net worth fell drastically to $94,267.85, representing a -5.73% ROI[2]. The agent executed over 61,000

[2]Note that per the figure, returns are defined as the percentage change in net worth value between consecutive periods, explicitly using simple returns. Transaction costs are clearly subtracted from gross returns to obtain net returns, ensuring clarity and interpretability in results.

Metric	No fees	Flat fee	Tiered fee	Buy-and-Hold
Final Net Worth $	108,330.79	94,267.85	101,421.02	108,306.96
RoI %	8.33	-5.73	1.42	8.31
Total Trades	59,391	61,727	62,699	1
Win Rate %	17.05	49.99	49.82	100
Fees Paid $	0.00	952,115.76	392,098.04	100.00
Sharpe Ratio	2.23	-3.20	0.79	1.71
Max Drawdown %	6.38	9.30	6.02	0.00
Calmar Ratio	1.31	-0.62	0.24	Inf

Table 2: Final model testing metrics.

Figure 3: Training results - Episode net worth (top left), episode rewards (top right), number of trades (bottom left) episode simple returns (bottom right).

trades, incurring $952,115.76 in fees. Even with a tiered fee model that reduced marginal costs at higher volumes, profitability did not come near to the No Fee strategy, with only a 1.42% ROI and $392,098.04 paid in fees. These findings reinforce that without incorporating transaction costs into the learning process or penalising excessive trading, RL agents may learn policies that are fundamentally uneconomical and unrealistic (Zhang et al., 2021).

Note however, this demonstrates the DDPG framework can effectively learn market patterns and execute profitable strategies when transaction costs are removed. The high Sharpe ratio for No Fee strategy (2.23) indicates good risk-adjusted returns and the Tiered Fee, the most realistic fee structure, has a fine risk-adjusted performance (Sharpe ratio: 0.79, Calmar ratio: 0.23).

Frequent trading in the model is also worth analysing - while the Tiered Fee strategy had much better performance (1.42% return) despite similar trading frequency (62,699 trades), with fees totalling $392,098 - about 40% of the Flat Fee model's costs. It seems that the Tiered Fee strategy quickly traded its way to the lower fee values due to high volumes and values over the model period. The Flat Fee strategy was left with consistent, proportionate trade fees.

Interestingly, while the agent's win rate improved under both fee scenarios (49.99% and 49.82%), this was not sufficient to offset the cost burden of frequent trading. In contrast, the Buy-and-Hold strategy, which made just one trade and paid only $100 in fees, maintained a competitive return of 8.31% with a perfect win rate, underscoring its efficiency in high-cost environments.

Note also that the critic loss decreased over the time period for all strategies from 140 - 170 to around 120 - 150. Steadily increasing Q-values over the training, from zero, to the 600 - 800 range, then peaking at the end of the training, indicate proper model training. Sadly, this didn't necessarily translate to better real-world performance.

3.2 Comparison to previous model run

Prior to running the final model, an initial model was run and hyperparameter tuning and cross validation steps were undertaken to statistically validate the model.

The training data was validated using a time-based cross-validation approach with three folds. Hyperparameter tuning was undertaken to identify the best neural network architecture and training parameters using grid search [3]. The buffer size of 50,000, sequence length (number of trading steps in buffer sequence) of 10, actor critic layer 1 and 2 neurons were left in the original configuration. Only the batch size (number of different sequences in each Sequential Experience Replay Buffer update) was decreased from 64 to 32 and updated for the final model training and testing run.

The updated training results, following cross-validation and hyperparameter tuning, showed clear improvements across most performance and risk metrics. The no-fee strategy achieved a significantly higher ROI (8.33% vs. 1.65%) and Sharpe ratio, indicating that the agent has learned a more effective trading policy. Additionally, the model executed fewer trades across all fee structures, suggesting improved trade selectivity. While the flat fee strategy remains unprofitable (–5.73% ROI), the loss is notably reduced from the previous –10.84%, and total fees paid have declined by around $50,000, indicating better cost control. Interestingly, the tiered fee model, which previously outperformed no-fee and flat fee setups, experienced a drop in ROI (from 4.83% to 1.42%), likely due to the model adopting a more conservative trading approach. Risk-adjusted returns also improved overall, with lower drawdowns and a stronger Calmar ratio for the no-fee model (1.31 vs. 0.21). These findings reinforce that while the agent effectively identifies profitable patterns, transaction costs continue to erode returns - particularly under flat or high-frequency fee structures. This highlights the importance of incorporating transaction-aware constraints into the training process to enhance real-world viability.

4 Discussion

The application of DDPG to the problem of optimising cryptocurrency trading strategies with fee constraints yielded positive yet nuanced results.

4.1 Critical interpretation of results

When evaluated across different transaction cost models, the agent demonstrated strong performance in an idealised, cost free environment. However, as expected, performance waned when introducing realistic trading fee conditions. This highlights the potential challenges of deploying RL in live financial markets. For example, in real markets, where trades are faced with margin calls, if a trading agent was on a particularly unprofitable run, margin calls would render these low win rate strategies useless.

As seen in the reviewed literature, RL models typically outperform a Buy-and-Hold benchmark—a trend mirrored in this study. For instance, Mastrogiovanni (2024) reported a Sharpe ratio of 1.51 for a model without trading penalties and 0.96 with penalties. It's important to note that in Mastrogiovanni's work, the Sharpe ratio was incorporated into the reward function, which may have influenced this strong Share ratio result. Similarly, Zhang et al. (2021) achieved Sharpe ratios of 0.49 (cost-free) and 0.60 (with $0.001 fee), using LSTM networks and a longer trading window. In comparison, this research achieved a No Fee strategy with a Sharpe ratio of 2.23, outperforming both sets of research, and Tiered Fee Share ratio of 0.79 which slightly outperformed the Zhang et al. version (though note

[3]The hyperparameters analysed included the actor and critic layer 1 size (64, 128 and 164 neurons), layer 2 size (32, 64, 96 neurons respectively) of the neural network, the buffer size (10000, 50000), sequence length (5, 10, 15) and batch size (32, 64, 128).

Zhang et al were optimising whole portfolios). Further research and testing would be needed to ensure that the model is not overfitting to the data (see below).

In this study, the agent's trading volume frequently pushed it into the more discounted fee tiers. When a flat-rate fee was applied, profitability dropped significantly, indicating the importance of cost structures in RL trading strategy performance. For the Tiered Fee strategy, as the agent was trading such significant values, it quickly reached the volume discount lower fees and reduced overall trading costs.

4.2 Limitations and future studies

In order to balance training efficiency and volume of training data, a short time period was analysed (15th August - 15th October). By increasing the time horizon and the number of assets analysed, the agent would have seen more significant swings in cryptocurrency prices and may have learned to generalise better than it has in this model (i.e., with the BTC price range of c. $66,000 - $51,000). In order to offset long training periods, early stopping could be implemented based on thresholds identified from this study. From a methodological perspective, DDPG's ability to operate in continuous action spaces offers clear advantages in financial settings, enabling granular position sizing (Lillicrap et al., 2015). Further analysis could be undertaken to test stationarity assumptions in time series modelling.

The analysis could also be improved by applying further cross validation and sensitivity analysis to the impact of fee tiers on trading performance. The test could be run with or without a trading frequency penalty to discourage frequent trading or other adjustments to smooth erratic trading activity. Running this test would identify a volume/ fee sweet spot for traders in the market.

In terms of improving the core DDPG framework - the current reward function, based on net wealth, was not normalised and ignored risk. As shown in Mastrogiovanni (2024), tailoring reward functions to incorporate Sharpe ratios or log returns can better align the agent's objective with investor utility, to prevent magnitude bias and promote robust decision-making similar to the Mastrogiovanni (2024) paper. A prioritised experience replay buffer (Schaul et al., 2016) could further enhance learning by sampling more frequently from high-priority experiences, where older experiences may no longer reflect current market conditions

Additionally, after the initial training run, regularisation was applied to the actor-critic neural networks and marginally improved performance across the board. Further regularisation and hyperparameter tuning could be applied using Optuna, a hyperparameter optimisation software (Tummon et al., 2020).

Finally, while the wider literature includes Buy-and-Hold strategies as baseline benchmarks, this analysis could additionally benchmark against traditional algorithmic strategies (e.g., momentum, trend-following) to establish a more robust performance evaluation framework.

It is worth noting that modelling reality remains a limitation across many RL models. As seen from the results, the agent makes many trades of varying sizes. In reality, market liquidity may impact the ability to place the high-volume of trades suggested by the actor. Other market microstructure aspects (high network fees in popular trading times or slippage in periods of lower liquidity) would impact our agents ability to realise profits.

4.3 Conclusion

The relevance of this work lies in its demonstration that RL agents can learn profitable policies in frictionless environments, validating prior studies (Tummon et al., 2020; Zhang et al., 2021). This work showed that including a tiered fee structure to the DDPG model can still yield positive ROI's and Sharpe ratios, however, the relative drop in performance under realistic cost structures exposes a lack of cost-awareness in training, resulting in economically inefficient behaviour.

The findings are limited by the exploration of hyperparameter space where only basic tuning and cross validation of actor-critic parameters was performed. Additionally, the focus on a short evaluation window, where results are based on a single asset (BTC/USD), limits its ability to generalise.

Overall, while DDPG demonstrates promise in learning profitable policies under ideal conditions, its practical viability in real-world trading hinges on integrating transaction cost awareness, reward shaping, and better regularisation.

5 Bibliography

Bhatia, A., Varakantham, P. & Kumar, A., 2019. Resource Constrained Deep Reinforcement Learning. *Proceedings of the International Conference on Automated Planning and Scheduling,* 29(1), pp. 610-620.

Binance, 2025. *Binance Fees & Transactions Overview.* [Online]
Available at: https://www.binance.com/en-GB/fee/trading [Accessed 23 March 2025].

Coinbase, 2025. *Coinbase Exchange Fees.* [Online]
Available at: https://help.coinbase.com/en/exchange/trading-and-funding/exchange-fees [Accessed 23 March 2025].

Deng, Y. et al., 2017. Deep Direct Reinforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems,* 28(3), pp. 653-664.

Felizardo, L. K., Paiva, F. C. L., Costa, A. H. R. & Del-Moral-Hernandez, E., 2022. Reinforcement Learning Applied to Trading Systems: A Survey. *arXiv,* cs.AI(2212.06064), p. https://arxiv.org/abs/2212.06064 .

Finance, C. A. M. L. i., 2025. *Tutorial: Deep Q-Learning for crypto trading - Workbook.* London: UCL.

Fowls, W., 2023. *Sharpe Ratio, Sortino Ratio, and Calmar Ratio.* [Online]
Available at: https://medium.com/@williamchristianfowls/sharpe-ratio-sortino-ratio-and-calmar-ratio-4738a3574fdb
[Accessed 25 March 2025].

Hayes, A., 2024. *Investopedia - Maximum Drawdown (MDD) Defined, With Formula for Calculation.* [Online] Available at: https://www.investopedia.com/terms/m/maximum-drawdown-mdd.asp
[Accessed 25 March 2025].

Kraken, 2025. *Kraken Fee Schedule.* [Online]
Available at: https://www.kraken.com/en-gb/features/fee-schedule#spot-crypto
[Accessed 23 March 2025].

Lillicrap, T. et al., 2015. Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Manfred Otto Heess, Tom Erez, Yuval Tassa, David Silver and Daan Wierstra. "Continuous control with deep reinforcement learning." CoRR abs/1509.02971 (2015): n. pag.. *CoRR,* Volume abs/1509.02971.

Liu, X.-Y., 2020. *FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance.* [Online]
Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3737257
[Accessed 25 March 2025].

Mastrogiovanni, M., July 01, 2024. *Reinforcement Learning Applied to Dynamic Portfolio Management: A Real Market Data Application,* s.l.: Available at SSRN: https://ssrn.com/abstract=4887926.
Mnih, V. e. a., 2015. Human-level control through deep reinforcement learning. *Nature,* Volume 518, p. 529–533.

Niklaus, P., 2025. *Blockpit.io - Bitcoin Halving Explained: What Investors Need to Know.* [Online]
Available at: https://www.blockpit.io/en-gb/blog/bitcoin-halving
[Accessed 25 March 2025].

Schaul, T., Quan, J., Antonoglou, I. & Silver, D., 2016. Prioritized Experience Replay. *https://arxiv.org/abs/1511.05952*.

Silver, D. et al., 2014. Deterministic Policy Gradient Algorithms. *Proceedings of the 31st International Conference on Machine Learning,* Volume 32.

Tummon, E., Adil Raja, M. & Ryan, C., 2020. Trading Cryptocurrency with Deep Deterministic Policy Gradients. In: C. Analide, P. Novais, D. Camacho & H. Yin, eds. *Intelligent Data Engineering and Automated Learning – IDEAL 2020: 21st International Conference, Guimaraes, Portugal, November 4–6, 2020, Proceedings, Part I.* Guimaraes: Springer, pp. 245-256.

Up, O. S., 2025. *Deep Deterministic Policy Gradient.* [Online]
Available at: https://spinningup.openai.com/en/latest/algorithms/ddpg.html#:~:text=transitions%2C%20%24B%20%3D%20%5C.one%20step%20of%20gradient%20ascent
[Accessed 24 March 2025].

Winder, P., 2020. *Reinforcement Learning.* 1st Edition ed. Sebastopol: O'Reilly Media, Inc.
Zhang, Z., Zohren, S. & Roberts, S., 2020. Deep Reinforcement Learning for Trading. *The Journal of Financial Data Science,* II(2), pp. 25-40.

Microsoft Copilot (version GPT-4 and 3.5Sonnet); Microsoft; OpenAI; Anthropic, Claude
Used for editing code and not original writing.